Ashish Panchal
Assignment 05
Assignment: Customer Debit Card Purchase Aggregation

Git Repo for the code:
https://github.com/panchalashish4/aws-glue-aggregation

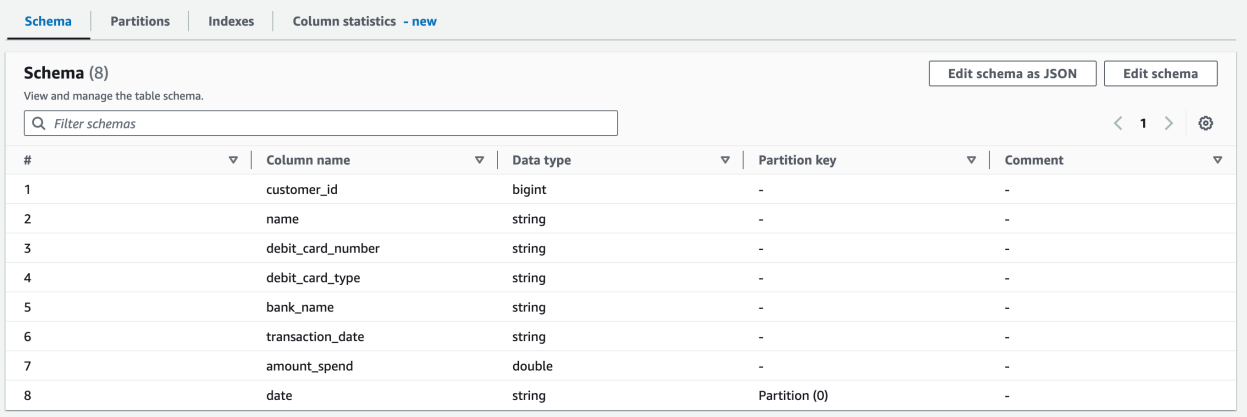
Python Script for Mock Data Generation

```
import csv
from random import randint, choice

# Define customer and bank names
customer_names = ["John Doe", "Jane Smith", "Michael Brown", "Sarah Lee",
"David Miller"]
bank_names = ["Citibank", "Chase", "Bank of America", "Wells Fargo", "PNC"]

# Define debit card types
card_types = ["Visa", "Mastercard"]

# Define number of transactions per day
transactions_per_day = 10

# Dictionary to store customer information (combined)
customer_info = {}


def generate_transaction(customer_name, current_date):
    """Generates a mock transaction record with a specific date"""
    if customer_name not in customer_info:
        # Generate card number and bank for new customer
        card_number = f"{randint(1000, 9999)}-{randint(1000, 9999)}-
{randint(1000, 9999)}-{randint(1000, 9999)}"
        bank_name = choice(bank_names)
        customer_info[customer_name] = {
            "customer_id": randint(1000000000, 9999999999),  # 10-digit
customer ID
            "debit_card_number": card_number,
            "debit_card_type": choice(card_types),
            "bank_name": bank_name,
        }

    # Use retrieved information and add customer name
    transaction_data = customer_info[customer_name].copy()
    transaction_data["name"] = customer_name
    # Set transaction date to the current day
    transaction_date = str(current_date)
    amount = round(randint(10, 100) + randint(0, 99) / 100, 2)  # Up to 2
decimal places
    transaction_data["transaction_date"] = transaction_date
    transaction_data["amount_spend"] = amount
    return transaction_data


def generate_transactions(num_transactions, current_date):
```

Ashish Panchal
Assignment 05
Assignment: Customer Debit Card Purchase Aggregation

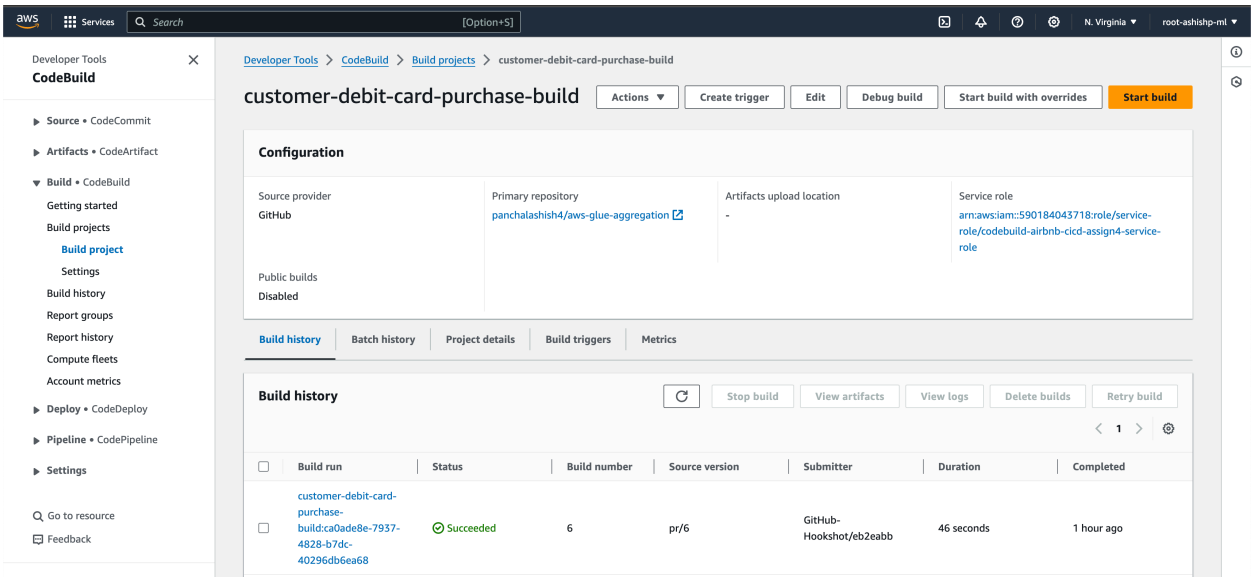
```
    """Generates a list of mock transaction records for a specific date"""
    transactions = []
    for _ in range(num_transactions):
        transactions.append(generate_transaction(choice(customer_names),
current_date))
    return transactions


def write_to_csv(data, filename):
    """Writes data to a CSV file"""
    with open(filename, "w", newline="") as csvfile:
        writer = csv.DictWriter(csvfile, fieldnames=["customer_id", "name",
"debit_card_number", "debit_card_type",
                                                     "bank_name",
"transaction_date", "amount_spend"])
        writer.writeheader()
        writer.writerows(data)
    return


def generate_data(current_date, date_str):
    """Generates data and creates csv files"""
    transactions = generate_transactions(transactions_per_day, current_date)
    write_to_csv(transactions, f"/tmp/transactions_{date_str}.csv")

    print(f"Generated mock transaction data transactions_{date_str}.csv and
saved in csv files")
    return
```

This code generates the mock data for customers.

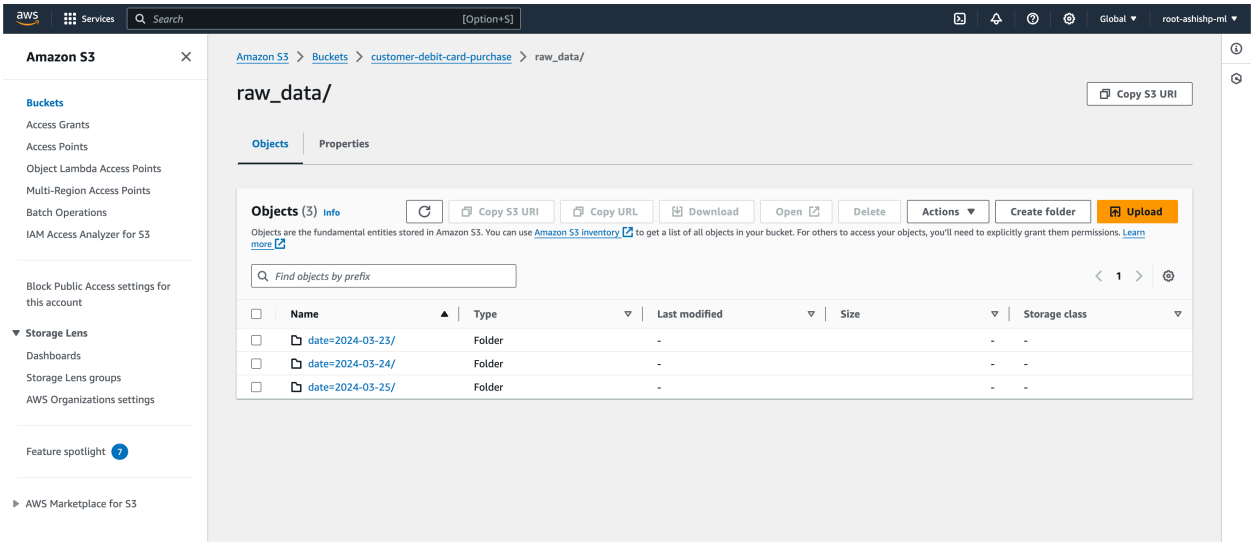


Ashish Panchal
Assignment 05
Assignment: Customer Debit Card Purchase Aggregation

CodeBuild for Lambda



S3 Bucket in Hive Partitioning:



Ashish Panchal
Assignment 05
Assignment: Customer Debit Card Purchase Aggregation

RDS Schema Code

```
import base64
import mysql.connector
import boto3
import json

# Initialize a Secrets Manager client
client = boto3.client('secretsmanager', region_name='us-east-1')
secret_name = 'mysql-rds-class5-creds'


def get_rds_credentials(secret_name):
    try:
        # Fetch the secret value
        get_secret_value_response =
client.get_secret_value(SecretId=secret_name)

        # Check if the secret uses the Secrets Manager binary field
        if 'SecretString' in get_secret_value_response:
            secret = get_secret_value_response['SecretString']
            secret_dict = json.loads(secret)
            return secret_dict
        else:
            decoded_binary_secret =
base64.b64decode(get_secret_value_response['SecretBinary'])
            secret_dict = json.loads(decoded_binary_secret)
            return secret_dict
    except Exception as e:
        print(f"Error fetching secret: {e}")
        return None


def connect_and_create_db():
    global password
    connection = None

    try:

        credentials = get_rds_credentials(secret_name)
        if credentials:
            print("Fecthed RDS Credentials:")
            username = credentials['username']
            password = credentials['password']
            # Depending on how you've structured your secret, you might need
            # to adjust the keys (e.g., 'username' and 'password')
accordingly.
        else:
            print("Failed to fetch credentials.")

        connection = mysql.connector.connect(
            host='mysql-aws-de-db-class5.cdcokcumq4ck.us-east-
1.rds.amazonaws.com',
            port=3306,
```

Ashish Panchal
Assignment 05
Assignment: Customer Debit Card Purchase Aggregation

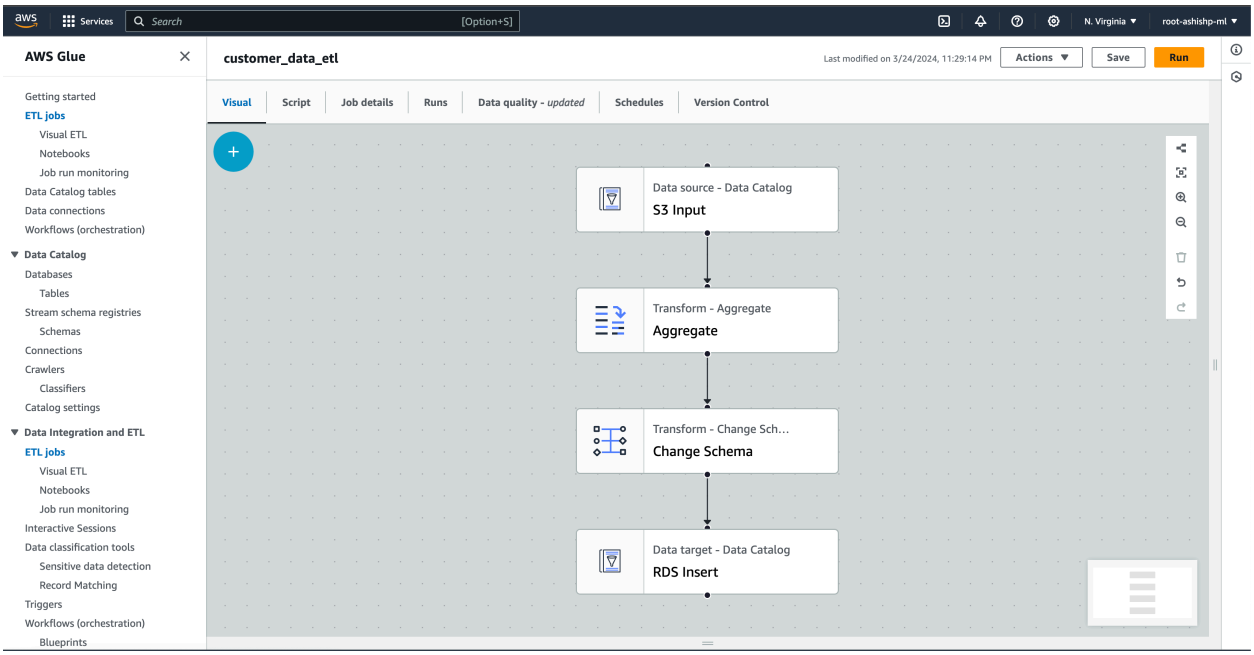
```
            user=username,
            password=password
        )

        if connection.is_connected():
            print("Successfully connected to the RDS instance.")

            cursor = connection.cursor()

            # Create a new database
            cursor.execute("create database if not exists customers;")
            print("Database created successfully.")

            cursor.execute("show databases;")
            print(cursor.fetchall())

            cursor.execute("use customers;")

            table_schema = """create table customer_transactions (
                                customer_id int not null,
                                debit_card_number varchar(255) not null,
                                bank_name varchar(255) not null,
                                total_amount_spend decimal(10,2) not null,
                                primary key (customer_id, debit_card_number,
bank_name));"""
            cursor.execute(table_schema)
            print("Table created successfully.")

            cursor.execute("show tables;")
            print(cursor.fetchall())

        else:
            print("Failed to connect to the RDS instance.")
    except mysql.connector.Error as e:
        print(f"Error: {e}")
    except Exception as e:
        print(f"Error: {e}")
    finally:
        if connection is not None and connection.is_connected():
            cursor.close()
            connection.close()
            print("Connection closed.")
    return
```

```
create table customer_transactions (
                customer_id int not null,
                debit_card_number varchar(255) not null,
                bank_name varchar(255) not null,
                total_amount_spend decimal(10,2) not null,
                primary key (customer_id, debit_card_number, bank_name));
```

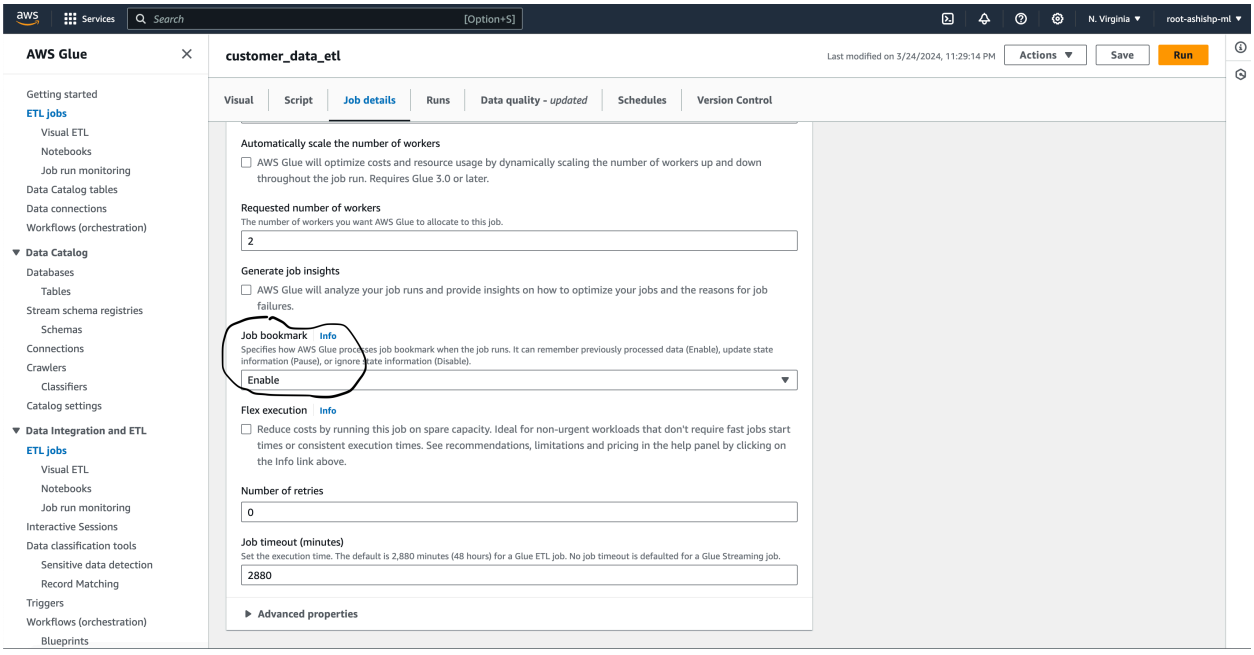
Ashish Panchal
Assignment 05
Assignment: Customer Debit Card Purchase Aggregation

This script will create database and table when lambda is executed. We'll create aggregate data by customer_id, debit_card_number and bank_name.
total_amount_spend will be the total amount they spend over time.

AWS Glue

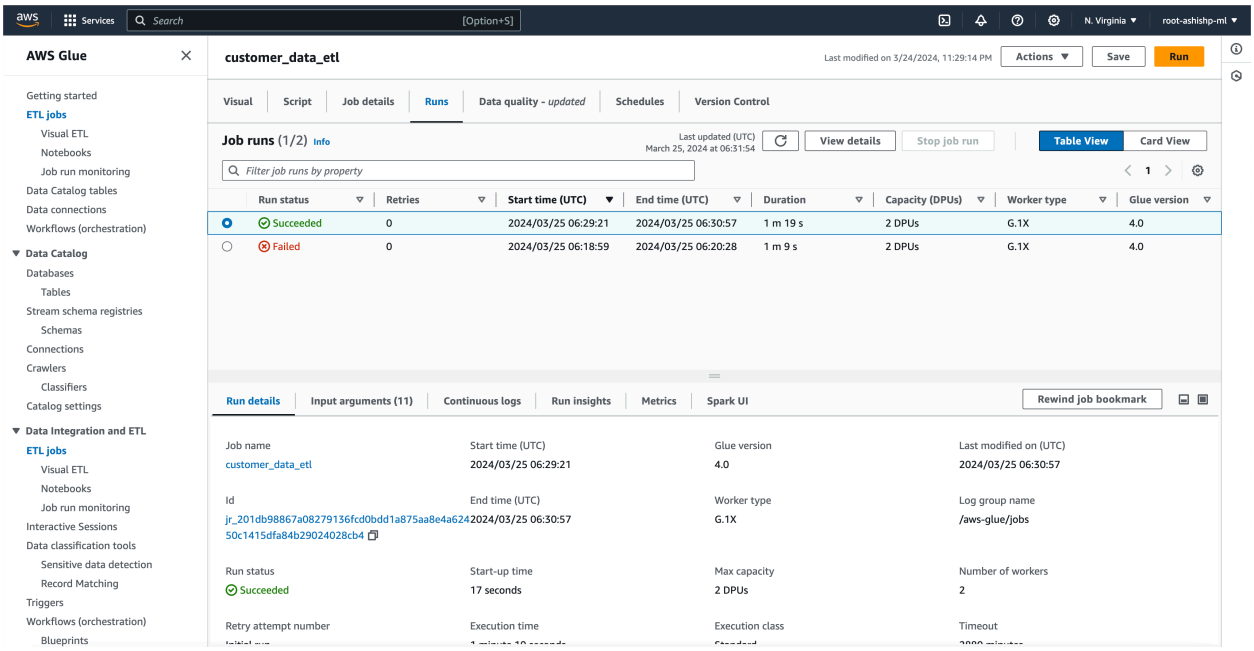


Incremental Load



Ashish Panchal
Assignment 05
Assignment: Customer Debit Card Purchase Aggregation

Glue Job Run



In this exercise, lambda_function is executed, which performs:

- **Generates mock_data and uploads csv files to s3 in hive partitioning format.**
- **Connect to RDS MySQL database, create new database and create customer_transactions table for aggregate data.**
- **AWS Glue job aggregates data from S3 and uploads into the RDS database.**